

Safe Trajectory Planning with Bernstein Polynomials Under Control Barrier Function Constraints

Jungeun Lee^{1,†}, Seongjae Lee^{1,†} and Jeong hwan Jeon^{1,2,*}

Abstract—Ensuring safe trajectory planning of autonomous mobile robots in environments with static and dynamic obstacles is a critical challenge for real-world deployment. Existing approaches, such as nonlinear model predictive control (NMPC) with control barrier functions (CBFs), can provide safety guarantees. However, their computational complexity scales with the prediction horizon, which can hinder real-world feasibility. We propose an NMPC algorithm that combines discrete-time CBFs with Bernstein polynomial parameterization. By constraining a constant number of Bernstein control points within a dynamically generated safe corridor, the resulting trajectory is guaranteed to remain forward invariant within the safe set. This formulation also reduces computational complexity compared with conventional approaches. Simulation results in both static and dynamic obstacle environments show that the proposed method achieves a high success rate while maintaining safety margins comparable to existing baselines. At the same time, computation time is significantly reduced and remains nearly constant as the prediction horizon increases. These results demonstrate the efficiency, robustness, and scalability of the proposed approach for safe trajectory planning in diverse scenarios.

I. INTRODUCTION

Recent advances in mobile robotics are driving the integration of autonomous systems into human environments. These systems are extending their influence from improving industrial productivity to enhancing convenience in daily life. In particular, the adoption of autonomous mobile robots (AMRs) is accelerating in industrial sectors such as logistics and manufacturing. The global market for AMRs was estimated at approximately \$4.07 billion in 2024 and is projected to grow to \$9.56 billion by 2030, corresponding to a compound annual growth rate (CAGR) of 15.1% [1]. The proliferation extends to the domestic sphere. More than half of U.S. households use smart-home technologies, creating an ecosystem that enables autonomous appliances such as robotic vacuum cleaners. As robots become increasingly widespread in our lives, ensuring their safety has emerged as a critical challenge. In particular, mobile robots responsible for transporting people or stuff are required to navigate safely

This work was supported by the InnoCORE program of the Ministry of Science and ICT (#N10250155), by the National Research Foundation of Korea (NRF) grant funded by the Korean government (MSIT) (No.RS-2022-NR070854), and by the Electronics and Telecommunications Research Institute (ETRI) grant funded by the Korean government [26ZD1160, Regional Industry ICT Convergence Technology Advancement and Support Project in Daegu-Gyeongbuk].

¹ Department of Electrical Engineering, Ulsan National Institute of Science and Technology (UNIST), Ulsan, Republic of Korea

² Artificial Intelligence Graduate School (AIGS), Ulsan National Institute of Science and Technology (UNIST), Ulsan, Republic of Korea

[†] These authors contributed equally. {jungeunlee, lsjj1999}@unist.ac.kr

* Corresponding author, jhjeon@unist.ac.kr

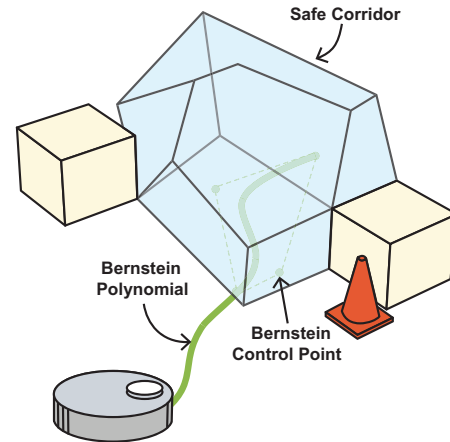


Fig. 1: Overview of the proposed NMPC approach. Safety is preserved by constraining Bernstein control points within a safe corridor, which guarantees forward invariance of the entire reference trajectory.

through environments containing both unpredictable dynamic obstacles and static obstacles.

To address collision avoidance, traditional methods such as the dynamic window approach (DWA) [2], timed elastic band (TEB) [3], and optimal reciprocal collision avoidance (ORCA) [4] have been widely used. DWA is a sampling-based method that searches for an optimal control input within the robot's dynamic constraints. In contrast, TEB [3] is an optimization-based approach that models the predicted trajectory as an elastic band, continuously deforming it to find a smooth and collision-free path. While these methods are effective in many scenarios, they lack guarantees of safety. These limitations have motivated the development of control barrier functions (CBFs) [5]. CBFs are founded on Lyapunov stability theory and are designed to ensure system safety by achieving forward invariance, which guarantees that the system state remains within a predefined safe set at all times. They have also been combined with model predictive control (MPC) to provide predictive safety and integrated with control Lyapunov functions (CLFs) to account for stability jointly. Furthermore, extending CBFs to discrete-time for practical implementation, known as DCBFs, has become a key area of research [6], [7]. Nevertheless, even with these extensions, standard CBFs still face several fundamental challenges. Especially for high-order dynamical systems where control inputs do not immediately affect the safety state, CBFs often result in overly cautious behaviors. In more severe cases, they lead to infeasibility, where no safe solution can be found. To enable robust handling of complex systems, high-order CBFs (HOCBFs) [8]–[10] have been developed by incorporating high-order derivative

constraints. A similar recent study [11], [12] explores the use of Bernstein polynomial parameterizations for motion planning. This approach emphasizes certifying reachable sets that reflect the capabilities of low-level tracking controllers, enabling long-horizon reasoning in hierarchical systems. In contrast, our work pursues a different safety objective. We integrate discrete-time CBFs into an NMPC formulated with Bernstein control points as decision variables. It directly enforces forward invariance of the entire reference trajectory within a safe set, while reducing the number of constraints.

We propose a novel NMPC framework for trajectory planning that ensures safety while significantly reducing computational complexity. Conventional approaches [5], [7]–[10] integrate CBFs into NMPC by enforcing safety constraints at every prediction step. In contrast, our method exploits the convex hull property of Bernstein polynomials to achieve safety with far fewer constraints. Restricting a constant number of Bernstein control points within a perception-driven safe corridor guarantees that the entire reference trajectory lies inside the safe set, as shown in Fig. 1. Under the convex hull property, the Bernstein reference trajectory remains within the safe corridor. The safety of the actual trajectory is then ensured under a bounded tracking error assumption through a tightened safe set. It is conceptually related to tube MPC [13] in robust control, where a nominal trajectory is constrained within a tightened set to guarantee safety of the actual trajectory under bounded disturbances. Beyond providing safety guarantees, we reduce the number of safety constraints from $O(mN)$ to $O(m(d+1))$ by exploiting the convex hull property of Bernstein polynomials. Here, m denotes the number of half-spaces defining the safe corridor, N is the predictive horizon length, and d is the Bernstein polynomial degree. As a result, safety verification becomes independent of the prediction horizon length, yielding substantial computational savings while preserving safety and robustness. The main contributions of this paper are threefold:

- We introduce an NMPC formulation parameterized by Bernstein control points that ensures safety while reducing the number of safety constraints.
- We provide a theoretical guarantee of robust safety under bounded tracking errors using tightened safe sets.
- Simulation results in static and dynamic environments show that the proposed method achieves high success rates and low infeasible rates with significantly reduced computation time compared to CBF-based MPC approaches [9], [10].

The remainder of this paper is organized as follows. Section II introduces the mathematical preliminaries. Section III details our proposed NMPC-based pipeline, focusing on how safety is formally guaranteed while reducing computational complexity through Bernstein control point parameterization. Section IV evaluates the proposed approach in diverse scenarios to demonstrate its efficiency and robustness. Finally, Section V concludes the paper and outlines directions for future research.

II. PRELIMINARIES

In this section, we introduce the concept of a CBF for discrete-time systems and define the Bernstein trajectory that ensures system safety.

Definition 2.1 (Discrete-Time Control Barrier Function [6]): Consider the discrete-time control system

$$\mathbf{x}_{k+1} = f(\mathbf{x}_k, \mathbf{u}_k), \quad k \in \mathbb{N}_{\geq 0}, \quad (1)$$

where $\mathbf{x}_k \in X \subset \mathbb{R}^n$ is the state and $\mathbf{u}_k \in U \subset \mathbb{R}^m$ is the control input at time step k . Let $h : X \rightarrow \mathbb{R}$ be a continuous function that defines the safe set

$$S := \{\mathbf{x} \in X | h(\mathbf{x}) \geq 0\}. \quad (2)$$

The continuous function h is a discrete-time control barrier function (CBF) for (1) with respect to the safe set (2) if there exists a class $\mathcal{K}$ function α such that for every state $\mathbf{x}_k \in X$, there exists a control input $\mathbf{u}_k \in U$ satisfying

$$h(f(\mathbf{x}_k, \mathbf{u}_k)) \geq h(\mathbf{x}_k) - \alpha(h(\mathbf{x}_k)). \quad (3)$$

Definition 2.2 (Safe Control Set [9]): For every state $\mathbf{x} \in X$, the safe control set is

$$C := \{\mathbf{u} \in U | h(f(\mathbf{x}, \mathbf{u})) \geq h(\mathbf{x}) - \alpha(h(\mathbf{x}))\}, \quad (4)$$

where α is a class $\mathcal{K}$ function with $\alpha(r) < r$ for all $r > 0$.

Definition 2.3 (Forward Invariance [8], [9]): For the system (1), a safe set S (2) is forward invariant if every trajectory that starts inside S never leaves it:

$$\mathbf{x}_0 \in S \Rightarrow \mathbf{x}_k \in S, \quad \forall k \in \mathbb{N}_{\geq 0}. \quad (5)$$

Remark 2.1: If h is a discrete-time CBF and, at every step k , the control input is in the safe control set $\mathbf{u}_k \in C(\mathbf{x}_k)$, then the safe set S is forward invariant.

Proof: For the system (1), assume $\mathbf{x}_0 \in S$ so that $h(\mathbf{x}_0) \geq 0$. Suppose $h(\mathbf{x}_k) \geq 0$ and choose $\mathbf{u}_k \in C(\mathbf{x}_k)$. Then, $h(\mathbf{x}_{k+1}) \geq h(\mathbf{x}_k) - \alpha(h(\mathbf{x}_k)) \geq 0$ since $\alpha(r) < r$ for all $r > 0$. Hence $\mathbf{x}_{k+1} \in S$. By induction, $h(\mathbf{x}_k) \geq 0$ for all $k \in \mathbb{N}_{\geq 0}$. Consequently, the safe set S is forward invariant. ■

Definition 2.4 (Discrete-Time Bernstein Trajectory): Let $N, d \in \mathbb{N}$. Given control points $\{\mathbf{p}_i\}_{i=0}^d \subset \mathbb{R}^n$, define the Bernstein basis

$$B_i^d(\tau) = \binom{d}{i} \tau^i (1-\tau)^{d-i}. \quad (6)$$

The degree- d Bernstein trajectory sampled over the prediction horizon N is

$$\mathbf{x}_k = \sum_{i=0}^d B_i^d(\tau_k) \mathbf{p}_i, \quad (7)$$

where τ_k is k/N for all $k \in \{0, \dots, N\}$. Each $\mathbf{x}_k$ is a convex combination of the control points since $B_i^d(\tau_k) \geq 0$ and $\sum_{i=0}^d B_i^d(\tau_k) = 1$. In particular, $\mathbf{x}_0 = \mathbf{p}_0$ and $\mathbf{x}_N = \mathbf{p}_d$.

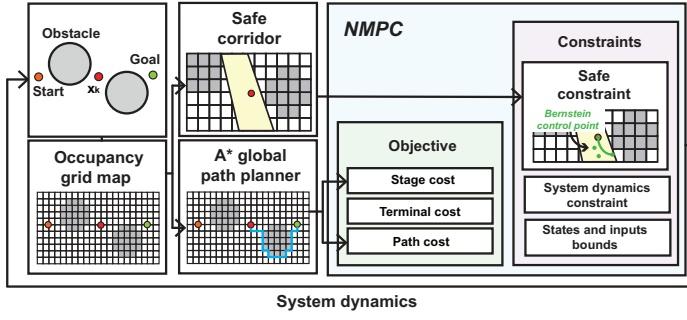


Fig. 2: NMPC-based collision-free trajectory planning and control pipeline. This framework ensures the entire reference trajectory remains in a safe set by constraining its Bernstein control points within a perception-driven corridor.

Algorithm 1 NMPC for collision-free trajectory planning and control with Bernstein control points.

Require: Sensor data stream $\mathbf{Z}_k$, Goal state $\mathbf{g}$, System dynamics $\mathbf{f}$, Maximum number of half-spaces defining the safe corridor m

- 1: **while** robot not at goal **do**
- 2: Initialize current state $\mathbf{x}_0$
- 3: Get an occupancy grid map $\mathbf{O}_k$ from $\mathbf{Z}_k$ and $\mathbf{x}_k$
- 4: Get a global path π_{global} from $\mathbf{O}_k$ and $\mathbf{x}_k$ using the A* algorithm
- 5: Construct the safe corridor defined by $\mathcal{S} = \bigcap_{i=1}^m \{\mathbf{x} | \mathbf{a}_i^T \mathbf{x} + \mathbf{b}_i \geq 0\}$ from $\mathbf{O}_k$ and $\mathbf{x}_k$
- 6: Get an optimal control $\mathbf{u}_k^* \leftarrow \text{NMPC}(\mathbf{x}_k, \pi_{\text{global}}, \mathcal{S})$
- 7: Apply the control input $\mathbf{u}_k^*$ and update the state $\mathbf{x}_k$
- 8: **end while**
- 9: **procedure** $\text{NMPC}(\mathbf{x}_k, \pi_{\text{global}}, \mathcal{S})$
- 10: Define the decision variables X , U , and P_{ctrl}
- 11: Minimize the cost (8) over the decision variables
- 12: subject to
- 13: system dynamics $\mathbf{x}_{k+1} = \mathbf{f}(\mathbf{x}_k, \mathbf{u}_k)$
- 14: state and input bounds
- 15: safety constraint (19)
- 16: **return** first optimal control input $\mathbf{u}_k^*$
- 17: **end procedure**

III. METHODOLOGY

This section details the formulation of a nonlinear model predictive control (NMPC) problem to ensure system safety. To achieve this, our pipeline, shown in Fig. 2 and Algorithm 1, follows a flow from perception to control. At each time step, our framework generates an occupancy grid map (OGM) by discretizing the sensor data into a grid of occupied and free cells. The A* algorithm then uses this map to plan a global path, omitting replanning for efficiency when the previous remains collision-free. Subsequently, a dynamic safe corridor is constructed from nearby obstacles to define a safe region, as shown in Fig. 3, following a safe corridor generation method in [9]. Each safe corridor is a convex region, defined as the intersection of half-spaces, $\mathcal{S} := \bigcap_{i=1}^m \{\mathbf{x} | \mathbf{a}_i^T \mathbf{x} + \mathbf{b}_i \geq 0\}$. The corresponding discrete-time CBF is then given by $h_i(\mathbf{x}) := \mathbf{a}_i^T \mathbf{x} + \mathbf{b}_i$, with the safe set condition $h_i(\mathbf{x}) \geq 0$ for all i .

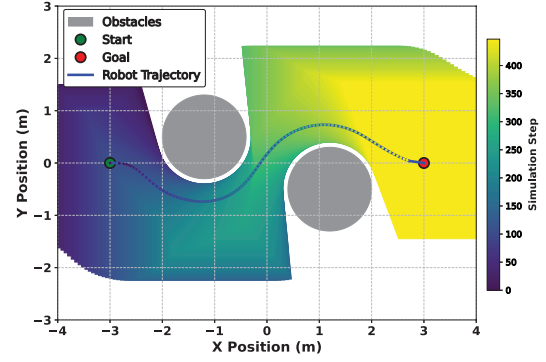


Fig. 3: Safe corridors accumulated over time steps. The safe corridors are dynamically reshaped by the mobile robot's position relative to nearby obstacles, ensuring safety throughout the trajectory with a predefined safety margin of 0.1 m.

We assume that the corridor hyperplanes are normalized, i.e., $\|\mathbf{a}_i\|_2 = 1$ for all i , so that $h_i(\mathbf{x})$ represents the signed distance to the corridor boundary. Finally, an optimal control problem is solved to find the control input that keeps the system state within the safe corridor. The remainder of this section will delve into our optimal control problem, which is formulated to find a safe local path and the corresponding control input required to follow it. For clarity, the formulations in this section are presented at the first time step, but they are recursively applied at each subsequent time step, as outlined in Algorithm 1.

A. Objective

The objective function achieves a balance among several competing goals, including state tracking, control effort, and input smoothness. Furthermore, it includes penalty terms to encourage global path guidance, reaching the goal, and minimizing slack variables. It is formulated to find the optimal control input which minimizes the cost function J at the first time step:

$$J(X, U, P_{\text{ctrl}}) = \sum_{k=0}^{N-1} L_k(\mathbf{x}_k, \mathbf{u}_k) + L_T(\mathbf{x}_N) + L_P(P_{\text{ctrl}}), \quad (8)$$

where $X = \{\mathbf{x}_0, \dots, \mathbf{x}_N\}$ is the predicted state sequence, $U = \{\mathbf{u}_0, \dots, \mathbf{u}_{N-1}\}$ is the control input sequence, and $P_{\text{ctrl}} = \{\mathbf{p}_0, \dots, \mathbf{p}_d\}$ is the set of Bernstein control points of a degree- d Bernstein polynomial. As shown in (8), the objective function is composed of three cost components.

- **Stage cost (L_k)** The term represents the sum of penalties for undesirable behaviors at each step k over the prediction horizon N . It is formulated as

$$L_k(\mathbf{x}_k, \mathbf{u}_k) = J_{\text{track},k} + J_{\text{ctrl},k} + J_{\text{smooth},k}. \quad (9)$$

The state tracking cost $J_{\text{track},k}$ penalizes deviations from a Bernstein reference trajectory parameterized by Bernstein control points:

$$J_{\text{track},k} = w_k (\mathbf{x}_k - \mathbf{x}_{r,k})^T \mathbf{Q} (\mathbf{x}_k - \mathbf{x}_{r,k}), \quad (10)$$

where $\mathbf{x}_k$ denotes the system state at step k , and $\mathbf{x}_{r,k}$ is its reference. w_k is a time-varying positive weight, and $\mathbf{Q}$ is a constant semi-definite weight. The control effort cost $J_{\text{ctrl},k}$

is a quadratic penalty on the deviation of the control input from its reference:

$$J_{\text{ctrl},k} = (\mathbf{u}_k - \mathbf{u}_{r,k})^T \mathbf{R} (\mathbf{u}_k - \mathbf{u}_{r,k}), \quad (11)$$

where $\mathbf{u}_k$ and $\mathbf{u}_{r,k}$ are the control input and its reference at step k , respectively. The weighting matrix $\mathbf{R} \succeq 0$ sets the relative penalty on each input channel. The input smoothness cost $J_{\text{smooth},k}$ penalizes changes in the control between consecutive steps:

$$J_{\text{smooth},k} = w_{\Delta u} \|\mathbf{u}_k - \mathbf{u}_{k-1}\|_2^2, \quad (12)$$

where $w_{\Delta u} \geq 0$ is a constant weight, and $\mathbf{u}_{-1}$ denotes the previously applied control input.

- **Terminal cost (L_T)** The terminal cost L_T is a Mayer term that strongly enforces convergence to the final reference state $\mathbf{x}_{r,N}$ at the end of the horizon:

$$L_T(\mathbf{x}_N) = (\mathbf{x}_N - \mathbf{x}_{r,N})^T \mathbf{P} (\mathbf{x}_N - \mathbf{x}_{r,N}), \quad (13)$$

where $\mathbf{P} \succeq 0$.

- **Path cost (L_P)** The path cost L_P serves to optimize both the shape and guidance of the Bernstein reference trajectory:

$$L_P(P_{\text{ctrl}}) = J_{\text{smooth}} + J_{\text{wp}} + J_{\text{goal}}. \quad (14)$$

The Bernstein path smoothness cost J_{smooth} penalizes the squared distance between consecutive control points to ensure a smooth path:

$$J_{\text{smooth}} = w_b \sum_{i=0}^{d-1} \|\mathbf{p}_{i+1} - \mathbf{p}_i\|_2^2, \quad (15)$$

where d is the degree of the Bernstein curve, and $\mathbf{p}_i \in \mathbb{R}^n$ denotes the i -th control point of the Bernstein curve. The waypoint guidance cost J_{wp} guides the Bernstein trajectory towards the waypoints sampled by the A* algorithm:

$$J_{\text{wp}} = w_{\text{wp}} \sum_{i=0}^d \min_j \|\mathbf{p}_{i,\text{pos}} - \mathbf{w}_j\|_2^2, \quad (16)$$

where $\mathbf{p}_{i,\text{pos}}$ is the position components of the i -th control point, and $\mathbf{w}_j$ is the j -th waypoint. The global attraction cost strongly attracts the final control point to the goal.

$$J_{\text{goal}} = w_{\text{goal}} \|\mathbf{p}_{d,\text{pos}} - \mathbf{g}\|_2^2, \quad (17)$$

where $\mathbf{g}$ is the local goal position. The weights w_b , w_{wp} , and w_{goal} are constants associated with (15), (16), and (17), respectively.

B. Constraints

The constraints ensure a physically feasible and safe trajectory by enforcing four conditions: system dynamics, bounds on states and control inputs, corridor-based safety, and geometric restrictions on the Bernstein control points. Specifically, the predicted trajectory must satisfy the discrete-time system dynamics $\mathbf{x}_{k+1} = \mathbf{f}(\mathbf{x}_k, \mathbf{u}_k)$, with all states and inputs remaining within the predefined bounds $\mathbf{x}_l \leq \mathbf{x}_k \leq \mathbf{x}_u$ and $\mathbf{u}_l \leq \mathbf{u}_k \leq \mathbf{u}_u$. Furthermore, the initial predicted state $\mathbf{x}_0$ must coincide with the robot's current measured state. In addition to the physical

and boundary constraints, safety is a critical requirement. The reference trajectory $\{\mathbf{x}_{r,k}\}_{k=0}^N$ parameterized by Bernstein control points is

$$\mathbf{x}_{r,k} = \sum_{j=0}^d B_j^d(\tau_k) \mathbf{p}_j, \quad (18)$$

where τ_k is k/N for all $k \in \{0, \dots, N\}$. The controller enforces safety by restricting the system state to a predefined safe corridor, mathematically expressed as the intersection of multiple linear inequalities. If the CBF is defined by m linear inequalities $\mathbf{a}_i^T \mathbf{x} + \mathbf{b}_i \geq 0$, $i = 1, \dots, m$, the constraints on each control point $\mathbf{p}_j$ are expressed in the relaxed form:

$$h_i(\mathbf{p}_j) - (1 - \gamma)h_i(\mathbf{x}_0) \geq 0, \quad \forall i \in \{1, \dots, m\}, \forall j \in \{0, \dots, d\}. \quad (19)$$

Here, $h_i(\cdot)$ is the CBF associated with the i -th half-space defining the safe corridor. The parameter $\gamma \in [0, 1]$ corresponds to a linear class- $\mathcal{K}$ function $\alpha(r) = \gamma r$ in Definition 2.2 and acts as a relaxation factor that relaxes the constraint near the boundary to improve feasibility [9]. Additionally, constraint (19) guarantees forward invariance of the Bernstein trajectory with respect to the safe corridor. Since the first control point coincides with the current state $\mathbf{p}_0 = \mathbf{x}_{r,0} = \mathbf{x}_0$, feasibility at the initial condition implies that all subsequent control points remain within the safe corridor. Due to the convex hull property of Bernstein polynomials, this further ensures that the entire reference trajectory is contained within the safe corridor.

Theorem 3.1 (Safety of Bernstein Trajectory): Let the safe corridor be defined as

$$\mathcal{S} := \bigcap_{i=1}^m \{\mathbf{x} | \mathbf{a}_i^T \mathbf{x} + \mathbf{b}_i \geq 0\}. \quad (20)$$

If the initial state coincides with the first control points, i.e., $\mathbf{p}_0 = \mathbf{x}_0$, and satisfies $h_i(\mathbf{x}_0) \geq 0$ for all $i = 1, \dots, m$, then the entire Bernstein trajectory remains within the safe corridor:

$$\mathbf{x}_{r,k} \in \mathcal{S}, \quad \forall k = 0, \dots, N. \quad (21)$$

Proof: By assumption, the initial state satisfies the safe corridor constraints, $h_i(\mathbf{x}_0) \geq 0$ for all i . From Eq. (19), each control point $\mathbf{p}_j$ of the Bernstein trajectory must satisfy

$$h_i(\mathbf{p}_j) \geq 0, \quad \forall j = 0, \dots, d. \quad (22)$$

Since each $h_i(\cdot)$ is affine, if all control points satisfy (22), then any convex combination of them must also satisfy $h_i(\mathbf{x}_{r,k}) \geq 0$ due to the convex hull property of Bernstein polynomials. Thus, we have $\mathbf{x}_{r,k} \in \mathcal{S}$ for all k . The entire Bernstein trajectory remains within the safe corridor. ■

Definition 3.1 (Tightened Safe Set under Tracking Error): Let $\mathbf{x}_k$ and $\mathbf{x}_{r,k}$ denote the actual system state and reference Bernstein trajectory at step k , respectively. Assume the tracking error is uniformly bounded:

$$\|\mathbf{x}_k - \mathbf{x}_{r,k}\|_2 \leq \epsilon, \quad \forall k = 0, \dots, N. \quad (23)$$

The ϵ -tightened safe set is

$$\mathcal{S}_{\Theta\epsilon} := \{\mathbf{x} \in \mathbb{R}^n | \mathbb{B}_\epsilon(\mathbf{x}) \subseteq \mathcal{S}\}, \quad (24)$$

where $\mathbb{B}_\epsilon(\mathbf{x}) := \{\mathbf{y} \in \mathbb{R}^n \mid \|\mathbf{y} - \mathbf{x}\|_2 \leq \epsilon\}$ denotes the closed Euclidean ball of radius ϵ centered at $\mathbf{x}$.

Theorem 3.2 (Robust Safety via Tightening): Suppose the Bernstein reference trajectory $\{\mathbf{x}_{r,k}\}_{k=0}^N$ lies entirely in the tightened safe set $\mathcal{S}_{\ominus\epsilon}$. If the tracking error is uniformly bounded by $\|\mathbf{x}_k - \mathbf{x}_{r,k}\|_2 \leq \epsilon$ for all k , then the actual trajectory also remains in the original safe set $\mathcal{S}$:

$$\mathbf{x}_k \in \mathcal{S}, \quad \forall k = 0, \dots, N. \quad (25)$$

Proof: By Definition 3.1, we have $\mathbb{B}_\epsilon(\mathbf{x}_{r,k}) \subseteq \mathcal{S}$ for every reference point $\mathbf{x}_{r,k}$. Since the tracking error is bounded by $\|\mathbf{x}_k - \mathbf{x}_{r,k}\|_2 \leq \epsilon$, it follows that $\mathbf{x}_k \in \mathbb{B}_\epsilon(\mathbf{x}_{r,k})$. Therefore, $\mathbf{x}_k \in \mathcal{S}$ for all k , which establishes the robust safety guarantee. ■

Remark 3.1 (Practical Implementation): While Theorem 3.2 establishes robust safety by requiring the reference trajectory to lie in the ϵ -tightened safe set, in practice we enforce a stronger sufficient condition that combines this tightening with the relaxed CBF structure in (19). Specifically, the implemented constraint takes the form

$$h_i(\mathbf{p}_j) - (1 - \gamma)h_i(\mathbf{x}_0) \geq \epsilon, \quad \forall i, j. \quad (26)$$

Since $h_i(\mathbf{x}_0) \geq 0$ for feasible initial states, this constraint implies $h_i(\mathbf{p}_j) \geq \epsilon$, ensuring that the Bernstein reference trajectory lies in the ϵ -tightened safe set.

A key advantage of the proposed approach is that safety verification needs to be performed only at the Bernstein control points, rather than at every step of the prediction horizon. In conventional methods [7]–[10], m safety constraints are imposed at each of the N prediction steps, resulting in a total of mN constraints and a computational burden that grows linearly with the horizon length. In contrast, our Bernstein trajectory requires only $m(d+1)$ constraints, where d is the degree of the Bernstein polynomial and typically $d \ll N$. This reduction transforms the safety check into a constant-size problem independent of the horizon length, thereby yielding a substantial computational saving while still guaranteeing forward invariance of the entire reference trajectory through the convex hull property.

IV. SIMULATION RESULTS

In this section, we demonstrate our approach through simulation experiments. Specifically, we show that the proposed NMPC framework ensures forward-invariant safety while substantially lowering computational costs. The results highlight not only improved feasibility in complex environments but also significant efficiency gains over two baselines. We refer to our proposed approach as BP-NMPC-DCBF, which stands for Bernstein polynomial parameterized NMPC with CBFs.

A. Overview

1) *System dynamics:* To validate the proposed approach, numerical case studies were carried out on a unicycle model in discrete time. The state vector is $\mathbf{x}_k = [l_{x,k}, l_{y,k}, \theta_k, v_k]^T$, which consists of position, heading, and linear speed. The

TABLE I: Cost function weights. Weighting matrices and parameters used in the control objective.

Parameter (Symbol)	Value
State weights ($\mathbf{Q}, \mathbf{P}$)	$\text{diag}(10^4, 10^4, 10^3, 10^2)$
Input weight ($\mathbf{R}$)	$\text{diag}(1, 1)$
Smoothness weight (w_b)	10
Waypoint guidance weight (w_{wp})	1
Goal attraction weight (w_{goal})	1000

control input is $\mathbf{u}_k = [\omega_k, a_k]^T$, denoting angular velocity and linear acceleration. With a sampling period of $\Delta t > 0$, the system evolves according to the following state transition equation:

$$\mathbf{x}_{k+1} = \mathbf{f}(\mathbf{x}_k, \mathbf{u}_k) \triangleq \begin{bmatrix} l_{x,k} + v_k \cos \theta_k \Delta t \\ l_{y,k} + v_k \sin \theta_k \Delta t \\ \theta_k + \omega_k \Delta t \\ v_k + a_k \Delta t \end{bmatrix}. \quad (27)$$

In the simulations, the state and control variables are subject to bounded constraints: $l_{x,k}, l_{y,k} \in [-4, 4]$ m, $\theta_k \in [-2\pi, 2\pi]$ rad, $v_k \in [-0.5, 2]$ m/s, $\omega_k \in [-\pi, \pi]$ rad/s, and $a_k \in [-2, 2]$ m/s².

2) *Simulation settings:* All experiments are conducted under the same simulation environment with a discrete time step of $\Delta t = 0.1$ s. A trial is considered successful if the robot reaches a goal region with radius 0.1 m within 1000 steps (100 s). A collision is recorded if the distance between the robot center and any obstacle center becomes smaller than the sum of their radii. The specific parameter values and weighting matrices used in the proposed approach are summarized in Table I.

3) *System configurations:* All experiments are conducted in an 8 m $\times$ 8 m workspace with initial state $\mathbf{x}_0 = [-3.0, -3.0, \pi/4, 0.0]^T$ and goal state $\mathbf{g} = [3.0, 3.0, \pi/4, 0.0]^T$. In the static scenarios, 20 circular obstacles are randomly generated with centers uniformly sampled from $[-3.5, 3.5] \times [-3.5, 3.5]$ m and radii in $[0.5, 1.0]$ m, as shown in Fig. 4a. In the dynamic scenarios, a set of 12 circular obstacles are generated with initial centers sampled from $[-3.5, 3.5] \times [-3.5, 3.5]$ m and radii in $[0.2, 0.5]$ m. Each obstacle moves with a constant velocity $\mathbf{v}^i$, whose components are uniformly sampled from $[-0.4, 0.4]$ m/s. The obstacle centers evolve according to $\mathbf{c}_{k+1}^i = \mathbf{c}_k^i + \mathbf{v}^i \Delta t$, as shown in Fig. 4b. Reflective boundary conditions keep obstacles within the workspace, and a protected region around the goal prevents obstacles

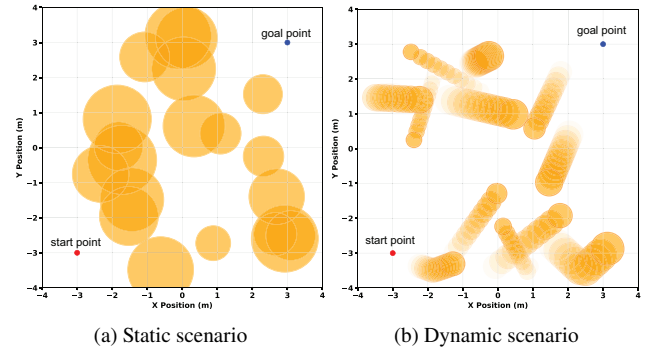


Fig. 4: Simulation environments. The static scenario contains randomly generated circular obstacles, while the dynamic scenario includes moving obstacles with randomly assigned velocities.

TABLE II: Performance comparison in static scenarios. The results are obtained from 500 randomized trials.

Algorithm	N	Success rate (%) $\uparrow$	Solve time (ms) $\downarrow$	Infeasible rate (%) $\downarrow$
iMPC (Order 2)	5	94.8	44.08 ± 3.78	0.17
	10	93.6	107.14 ± 5.85	0.31
	15	90.6	177.23 ± 8.78	0.52
iMPC (Order 1, predictive)	5	97.0	27.41 ± 2.84	0.09
	10	95.2	59.62 ± 5.26	0.21
	15	92.2	94.39 ± 8.68	0.48
NMPC-DCBF	5	100.0	8.24 ± 0.14	0.00
	10	100.0	13.95 ± 0.17	0.00
	15	99.8	20.61 ± 0.69	0.00
BP-NMPC-DCBF (Ours)	5	99.6	9.15 ± 0.23	0.00
	10	99.8	13.18 ± 0.30	0.00
	15	100.0	18.17 ± 0.28	0.00

from entering the safety radius. For the proposed method, the Bernstein polynomial degree is $d = 4$, the CBF decay rate is $\gamma = 0.5$, and the safety margin is 0.05 m. Each experiment is repeated for 500 randomized trials. A trial is considered successful if the agent reaches within 0.10 m of the goal without collision before the time limit.

B. Baselines

We compare the proposed BP-NMPC-DCBF with three baselines. The primary baseline is the first-order NMPC-DCBF [7]. We additionally evaluate two variants of the iterative MPC (iMPC) framework. The first variant, iMPC (Order 2), originally termed iMPC-DHOCBF [9], employs a second-order CBF based solely on current obstacle measurements. The second variant, iMPC (Order 1, predictive), originally termed iMPC-CBF-JPS [10], uses a first-order CBF incorporating predicted obstacle motion. To ensure a fair comparison, all baselines are integrated with the same A* global planner used in our framework, replacing the jump point search (JPS) planner used in [10].

C. Implementation Details

All control algorithms and simulations were implemented in Python 3.8.10. The nonlinear optimization problems for the proposed method and the NMPC-DCBF baseline were solved with IPOPT, whereas the quadratic programming (QP) problems for the iMPC baselines were solved with OSQP. All simulations were conducted on a PC equipped with an AMD Ryzen 9 7950X processor.

D. Results

Quantitative performance is evaluated using three key metrics. The success rate measures the ratio of successful trials to the total number of attempts. The computation time is reported as the average solver time per control step together with its standard deviation. The infeasible rate indicates the percentage of time steps in which the optimization solver failed to find a feasible solution relative to the total number of steps executed in successful scenarios.

The quantitative results for the static and dynamic environments are summarized in Table II and III. We evaluate the performance across prediction horizons $N \in \{5, 10, 15\}$. The

TABLE III: Performance comparison in dynamic scenarios. The results are obtained from 500 randomized trials.

Algorithm	N	Success rate (%) $\uparrow$	Solve time (ms) $\downarrow$	Infeasible rate (%) $\downarrow$
iMPC (Order 2)	5	80.8	39.17 ± 3.78	0.46
	10	81.4	94.70 ± 8.22	0.56
	15	81.2	168.21 ± 8.68	0.56
iMPC (Order 1, predictive)	5	85.4	25.61 ± 2.70	0.38
	10	85.8	59.31 ± 6.04	0.51
	15	86.4	90.91 ± 8.74	0.46
NMPC-DCBF	5	69.6	8.20 ± 0.44	1.11
	10	81.6	20.31 ± 0.79	0.53
	15	84.6	27.11 ± 1.23	0.57
BP-NMPC-DCBF (Ours)	5	92.4	9.79 ± 0.27	0.21
	10	88.8	14.05 ± 0.40	0.28
	15	88.0	18.77 ± 0.39	0.28

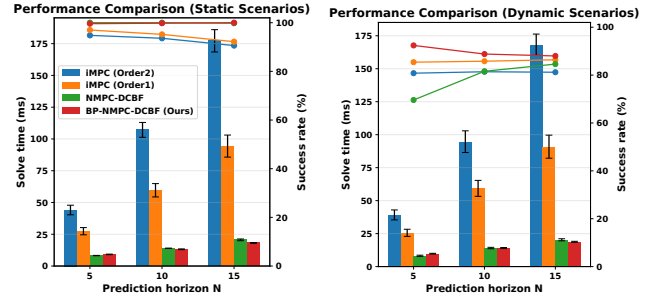


Fig. 5: Performance comparison across prediction horizons. Solve time (bar, left axis) and success rate (line, right axis) are shown for the baseline and proposed approaches in static and dynamic scenarios.

most prominent difference among the methods is the computational cost. As shown in both tables, the iMPC-based baselines exhibit substantially higher computation times that increase rapidly with the prediction horizon. For example, in the static scenarios with $N = 15$, iMPC (Order 2) requires 177.23 ms per step, whereas the proposed BP-NMPC-DCBF requires only 18.17 ms. This order-of-magnitude difference indicates that the iMPC baselines are less suitable for high-frequency real-time control. In the dynamic scenarios, the proposed method demonstrates clear advantages, particularly at short horizons. As shown in Table III, the NMPC-DCBF baseline suffers a substantial performance drop at $N = 5$, achieving only 69.6% success, whereas BP-NMPC-DCBF reaches 92.4%. In addition, the proposed method consistently maintains the lowest infeasible rates across all horizons, indicating improved numerical stability. Overall, these results indicate that the proposed BP-NMPC-DCBF approach significantly reduces computational cost compared to baselines while maintaining strong robustness in dynamic scenarios, as shown in Fig. 5.

Figs. 6 and 7 illustrate real-time executions of the proposed approach. The gray polygon denotes the safe corridor used to enforce the NMPC constraints, while the dashed curve shows the global guidance path generated by the A* algorithm. The proposed method computes Bernstein control points (green markers) that define the reference trajectory (solid green curve), which results in the robot trajectory (solid blue curve). Consistent with the quantitative results, the trajectories remain effective in both scenarios, maintaining feasibility and reliable trajectory planning under complex obstacle environments.

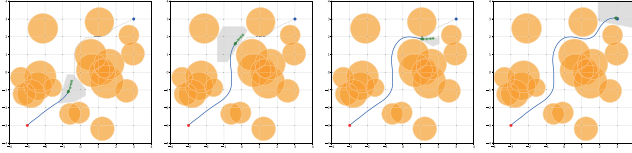


Fig. 6: Snapshots of the proposed BP-NMPC-DCBF in a static scenario. The environment contains 20 static obstacles, and snapshots are taken every 50 steps with $N = 5$ and $\gamma = 0.5$.

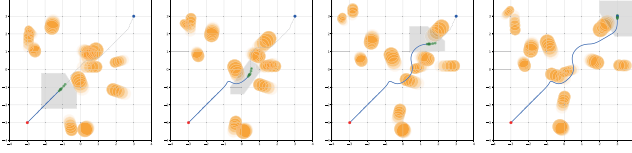


Fig. 7: Snapshots of the proposed BP-NMPC-DCBF in a dynamic scenario. The environment contains 12 static obstacles, and snapshots are taken every 25 steps with $N = 5$ and $\gamma = 0.5$.

E. Ablation Study

As shown in Fig. 8, the proposed BP-NMPC-DCBF consistently lies in the high-performance top-right region, achieving both high computational efficiency and strong safety. In contrast, NMPC-DCBF-based methods are computationally efficient but tend to lose safety at short horizons, whereas iMPC-based methods provide relatively higher safety but incur high computational costs. The results further indicate that the order of the safety constraint has a greater impact on performance than the inclusion of future obstacle information. In particular, second-order CBF constraints often underperform first-order counterparts due to excessive conservatism, which can lead to infeasible optimization problems in dynamic environments.

V. CONCLUSIONS

The paper proposes a nonlinear model predictive control (NMPC) algorithm parameterized by Bernstein control points with CBF constraints, termed BP-NMPC-DCBF. The central contribution lies in reducing the number of safety constraints. This reduction decouples safety verification from the prediction horizon length, yielding significant computational savings without compromising safety guarantees.

Nevertheless, the proposed framework is restricted to convex obstacles. Since the safety constraints are constructed based on hyperplane representations of obstacle boundaries, the method is not readily applicable to nonconvex obstacles. Extending the proposed approach to account for nonconvex obstacle geometries constitutes an important direction for future research.

REFERENCES

- [1] Grand View Research, "Autonomous mobile robots market size, share & trends analysis report," <https://www.grandviewresearch.com/industry-analysis/autonomous-mobile-robots-market>, Jan 2024, accessed: 2025-09-25.
- [2] D. Fox, W. Burgard, and S. Thrun, "The dynamic window approach to collision avoidance," *IEEE Robotics Automation Magazine*, vol. 4, no. 1, pp. 23–33, 1997.
- [3] C. Rösmann, W. Feiten, T. Wösch, F. Hoffmann, and T. Bertram, "Trajectory modification considering dynamic constraints of autonomous robots," in *ROBOTIK 2012; 7th German Conference on Robotics*. VDE, 2012, pp. 1–6.

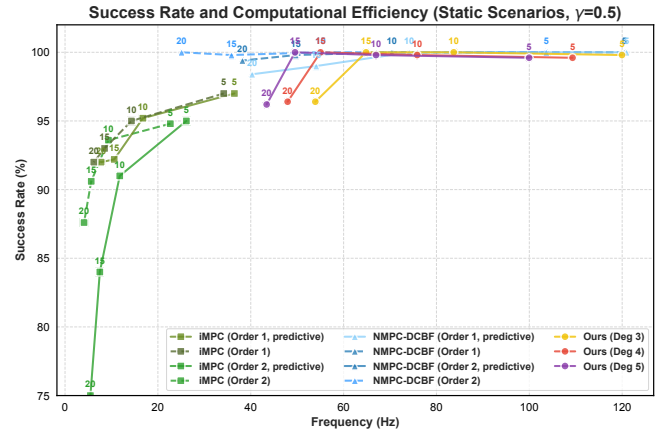
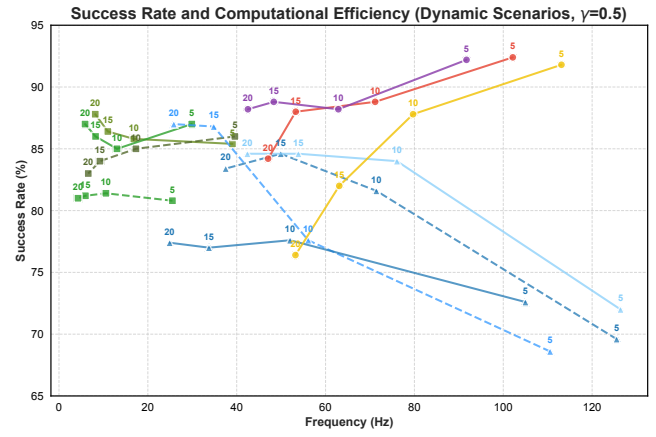


Fig. 8: Performance trade-off between success rate and computational efficiency. Success rate is plotted against control update frequency for different prediction horizons N in dynamic (top) and static (bottom) environments, comparing the baselines and proposed methods.

- [4] J. van den Berg, S. J. Guy, M. Lin, and D. Manocha, "Reciprocal n-body collision avoidance," in *Proc. Int. Symp. Robotics Research (ISRR)*, 2011, pp. 3–19.
- [5] W. Xiao, C. G. Cassandras, and C. Belta, *Safe autonomy with control barrier functions: theory and applications*. Springer, 2023.
- [6] Y. Xiong, D.-H. Zhai, M. Tavakoli, and Y. Xia, "Discrete-time control barrier function: High-order case and adaptive case," *IEEE Transactions on Cybernetics*, vol. 53, no. 5, pp. 3231–3239, 2022.
- [7] J. Zeng, Z. Li, and K. Sreenath, "Enhancing feasibility and safety of nonlinear model predictive control with discrete-time control barrier functions," in *2021 60th IEEE Conference on Decision and Control (CDC)*. IEEE, 2021, pp. 6137–6144.
- [8] W. Xiao and C. Belta, "Control barrier functions for systems with high relative degree," in *2019 IEEE 58th conference on decision and control (CDC)*. IEEE, 2019, pp. 474–479.
- [9] S. Liu, J. Zeng, K. Sreenath, and C. A. Belta, "Iterative convex optimization for model predictive control with discrete-time high-order control barrier functions," in *2023 American Control Conference (ACC)*, 2023, pp. 3368–3375.
- [10] S. Liu, Y. Mao, and C. A. Belta, "Safety-critical planning and control for dynamic obstacle avoidance using control barrier functions," in *2025 American Control Conference (ACC)*. IEEE, 2025, pp. 348–354.
- [11] N. Cosmay-Shanklin and A. D. Ames, "Bézier reachable polytopes: Efficient certificates for robust motion planning with layered architectures," in *2025 American Control Conference (ACC)*, 2025, pp. 5052–5059.
- [12] J. Park, J. Kim, I. Jang, and H. J. Kim, "Efficient multi-agent trajectory planning with feasibility guarantee using relative bernstein polynomial," in *2020 IEEE International Conference on Robotics and Automation (ICRA)*. IEEE, 2020, pp. 434–440.
- [13] D. Mayne, M. Seron, and S. Raković, "Robust model predictive control of constrained linear systems with bounded disturbances," *Automatica*, vol. 41, no. 2, pp. 219–224, 2005.